

# Validação Operacional B-39 — Fila de Webhooks

Retry, Backoff, Dead Letter Queue, Idempotência e Resiliência

Relatório Técnico de Validação em Ambiente Real  
Docker local · Redis real · Postgres real · Bull real

Data: 29 de julho de 2026  
Documento: VALIDACAO\_OPERACIONAL\_B39\_ATENDEHUB.pdf

# Sumário

---

- 1. Resumo executivo
- 2. Objetivo
- 3. Escopo
- 4. Ambiente de testes
- 5. Arquitetura analisada
- 6. Fluxograma atual (antes x depois)
- 7. Validações executadas (V1-V11)
- 8. Achado crítico: DLQ prematura (encontrado e corrigido nesta sessão)
- 9. Achado alto: race condition de idempotência (encontrado, não corrigido)
- 10. Auditoria de código
- 11. Testes automatizados (resultado final)
- 12. Métricas e tempos coletados
- 13. Riscos residuais
- 14. Classificação de severidade consolidada
- 15. Recomendações e melhorias futuras
- 16. Checklist Production Ready
- 17. Conclusão final e aprovação da arquitetura

# 1. Resumo Executivo

---

Esta validação executou uma bateria de 11 cenários operacionais contra a implementação real do B-39 (retry/backoff/DLQ/idempotência da fila de webhooks), usando infraestrutura genuína (Docker local, Redis real, Postgres real, Bull real) em vez de apenas suíte de testes unitários. O objetivo era comprovar — não presumir — que a correção anterior do B-39 tornou o sistema resiliente.

O exercício encontrou e corrigiu, durante a própria validação, um BUG CRÍTICO não detectado pelos testes unitários anteriores: o WebhookProcessor movia qualquer falha transitória para a Dead Letter Queue já na 1ª tentativa, mesmo quando o Bull ainda ia retentar com sucesso — poluindo a DLQ com falsos positivos. A causa raiz (o evento "failed" do Bull dispara em toda tentativa, não só na última) foi confirmada lendo o código-fonte do Bull, corrigida com uma guarda de 3 linhas, e re-validada com sucesso na mesma sessão.

Um segundo achado, de severidade alta e AINDA NÃO CORRIGIDO, foi encontrado: sob concorrência real (não sequencial), o mecanismo de deduplicação de mensagens tem uma condição de corrida — `Message.externalId` não tem constraint única no banco. Isso não é o bug original do B-39 (é justamente o retry voltando a funcionar que expõe essa lacuna), mas é relevante para o veredito final.

Com a correção da DLQ prematura aplicada, todos os demais cenários testados — graceful shutdown, idempotência sequencial, matriz de resiliência (500/502/503/504/429/timeout/ECONNRESET/DNS/ETIMEDOUT), saturação de fila, indisponibilidade e recuperação do Redis, e restart completo — passaram com evidência real.

**Ø=Bá Veredito: APROVADO COM RESSALVAS — ver seção 17**

# 2. Objetivo

---

Verificar, com evidência técnica real (não suposição), se a correção do B-39 garante: nenhuma mensagem perdida silenciosamente, retry automático funcional, backoff exponencial correto, Dead Letter Queue íntegra e reprocessável, idempotência sob reentrega, logs estruturados completos, métricas corretas, e resiliência a falhas de infraestrutura (Redis, worker, rede).

# 3. Escopo

---

Incluído: fila ``webhook`` e ``webhook-dlq``, WebhookProcessor, WebhookService (idempotência), WebhookDlqService, WebhookMetricsService, classificação de erros, backoff customizado, comportamento do Bull/Redis sob falha real.

Fora do escopo desta rodada: carga sintética acima de ~2.600 jobs (extrapolada, não executada — ver seção 12); endpoint HTTP ``/webhooks/metrics`` exercitado ao vivo via requisição real (coberto só por teste automatizado); circuit breaker (não existe — avaliado tecnicamente, não implementado).

# 4. Ambiente de Testes

---

| Componente       | Detalhe                                                                             |
|------------------|-------------------------------------------------------------------------------------|
| Docker           | Docker Desktop (Windows) — iniciado nesta sessão                                    |
| Postgres         | atendehub_postgres — container real, healthy                                        |
| Redis            | atendehub_redis — container real, AOF habilitado ( <code>--appendonly yes</code> )  |
| Fila de teste    | b39-validation / b39-validation-dlq (descartável — nunca webhook/webhook-dlq reais) |
| Empresa de teste | b39-validation (Postgres real, removida ao final)                                   |
| Harness          | apps/api/tmp-validation/*.ts — usa as classes reais do B-39, não reimplementações   |

# 5. Arquitetura Analisada

---

Controller (HTTP, ack imediato) !' Queue "webhook" (Bull/Redis) !' Processor (classifica erro, decide retry/discard) !' Service (Contact/Conversation/Message, idempotente por externalId) !' em falha definitiva !' DLQ "webhook-dlq" (sem processor, só acumula) !' reproprocessamento manual via WebhookDlqController (isolado por tenant).

## 6. Fluxograma Atual (pós-correção desta sessão)

---

```
Controller --200 rápido--> Queue.add(buildWebhookJobOptions(ENV))
|
Processor: classifica erro (transient/permanent/unknown)
-> permanent: job.discard()
-> throw sempre
|
Service.handleEvent: log + throw (nunca engole) [correção B-39 original]
|
Bull: job failed -> "failed" event dispara SEMPRE (toda tentativa)
|
onFailed: willRetry = attemptsMade < maxAttempts && !isDiscarded()
-> true:  RETORNA (nao mexe na DLQ) [correcao desta sessao]
-> false: DLQ + Sentry + metrica
|
ADMIN reprocessa via /webhooks/dlq (isolado por tenant, dedupe por externalId)
```

## 7. Validações Executadas

---

### V1 — Graceful Shutdown

Worker real processando um job de 25s foi morto com ``taskkill /F`` (10s dentro da execução, bem antes do fim natural). Job permaneceu íntegro no Redis (sem `finishedOn`, presente em "active", lock órfão). Um worker #2 novo detectou o job travado via `stalled-check` do Bull (~26s) e o reprocessou DO ZERO com sucesso.

' **Job íntegro, lock liberado, job volta pra fila, retry acontece, nada perdido**

Achado técnico: recuperação de job travado é sempre um reprocessamento do zero, não uma retomada — reforça a dependência direta na idempotência (V2/V3). Ressalva: SIGTERM emulado do Node (cross-processo) não terminou o processo no 1º teste; `taskkill /F` foi determinístico. Sem `app.enableShutdownHooks()` em `main.ts` hoje, um SIGTERM real de orquestrador em produção mataria o processo sem handler — o comportamento observado é exatamente o que ocorreria hoje.

### V2/V3 — Idempotência e Partial Failure

5 entregas SEQUENCIAIS do mesmo `externalId` via `WebhookService` real + Postgres real: 1 único `Contact/Conversation/Message` criados, `emitNewMessage` e auto-atendimento disparados apenas 1x — confirmado por SELECT direto no banco.

' **Idempotência sequencial: PASSOU**

5 entregas CONCORRENTES (`Promise.all`) do mesmo `externalId`: 2 linhas de `Message` duplicadas no banco, `emitNewMessage` e `handleReply` disparados 2x.

Ø=⚡ Idempotência sob concorrência real: FALHOU — achado alto, não corrigido (ver seção 9)

### V6/V7 — DLQ e Matriz de Resiliência

Bateria com HTTP 500/502/503/504/429, timeout, ECONNRESET, DNS/ENOTFOUND, ETIMEDOUT (todos configurados para falhar 1x e recuperar na 2ª tentativa) + 2 cenários forçando DLQ (erro permanente, esgotamento de tentativas), contra o `WebhookProcessor` real.

Ø=⚡ 1ª rodada: FALHOU — bug crítico de DLQ prematura encontrado (ver seção 8)

' 2ª rodada (após correção): PASSOU — 0 falsos positivos, DLQ só recebe falha definitiva

### V4/V5 — Observabilidade e Métricas

Campos `requestId/tracelId/attempt/processor/queue/worker/elapsedTime` presentes em todos os logs de tentativa; `tenantId/conversationId/messageId` presentes só no log de falha definitiva (achado médio, decisão de trade-off latência x observabilidade, documentado). Contadores de `WebhookMetricsService` evoluem corretamente e batem com a contagem manual da bateria de testes (dlq:13 com bug -> dlq:2 após correção).

Ø=⚡ Correlação ponta a ponta funciona; cobertura de campo parcial fora do caminho de falha

### V8 — Saturação da Fila

100 / 600 / 1.600 / 2.600 jobs processados sem degradação de memória (worker: 436MB->449MB) nem de vazão. 5.000/10.000 extrapolados, não executados fisicamente (decisão consciente de tempo — gargalo real de produção é Postgres/Evolution por job, não a fila).

' **Sem gargalo na camada de fila/Redis; concorrência 1 é o teto real (achado arquitetural)**

### V9 — Indisponibilidade do Redis

Container Redis local derrubado (``docker stop``) com worker ativo. Produzir trava indefinidamente ao tentar enfileirar durante a queda (achado médio: sem fail-fast). Após religar o Redis, o MESMO processo worker (nunca reiniciado) reconectou sozinho e processou o próximo job sem intervenção.

' **Reconexão automática, integridade preservada, zero jobs perdidos**

### V10 — Recuperação após Restart Completo

Jobs enfileirados sem worker ativo, Redis reiniciado por completo (``docker restart``), depois um worker

TOTALMENTE NOVO processou tudo corretamente (sucesso + DLQ). Entrada da DLQ confirmada íntegra através de outro restart do Redis.

' **Jobs, fila e DLQ preservados através de restart completo**

## **V11 — Circuit Breaker**

Confirmado por busca no código: não existe circuit breaker implementado. Avaliação: vale como melhoria (não bloqueante) em torno do EvolutionService — retry+DLQ já dão resiliência por job, mas não resiliência agregada (nada detecta "Evolution caiu" uma vez e para de tentar globalmente). Biblioteca recomendada: cockatiel (TS nativo, zero dependências) ou opossum (mais tradicional).

**Ø=Não implementado — recomendação registrada para o roadmap**

## 8. Achado Crítico: DLQ Prematura (Encontrado e Corrigido Nesta Sessão)

---

### Ø=Ý4 Severidade: CRÍTICA

Causa raiz: o evento "failed" do Bull dispara em TODA tentativa que lança uma exceção, não só na última — confirmado lendo `node_modules/bull/lib/queue.js` (`handleFailed` chama `job.moveToFailed(err)` e emite "failed" incondicionalmente; é o próprio `moveToFailed` que decide, internamente, se agenda `retry` ou marca falha terminal). O `WebhookProcessor#onFailed` das sessões anteriores do B-39 assumia — por comentário explícito no código, nunca verificado contra o comportamento real — que esse evento só dispararia na tentativa final.

Efeito: toda falha transitória (mesmo uma que o próprio `retry` resolveria sozinho) criava uma entrada na Dead Letter Queue, disparava alerta no Sentry e incrementava a métrica de DLQ — poluindo a fila de falhas com falsos positivos e gerando ruído operacional desnecessário.

Correção aplicada: guarda no início de `onFailed` — ``willRetry = job.attemptsMade < maxAttempts && ! job.isDiscarded()``; retorna imediatamente sem tocar DLQ/Sentry/métrica quando o Bull ainda vai retentar.

Evidência de correção: bateria de 9 cenários transitórios re-executada — 0 entradas de DLQ (antes: 9 falsas). 1 novo teste de regressão adicionado a `webhook.processor.spec.ts`. Suíte completa: 346/346 (era 345/345 antes desta sessão).

## 9. Achado Alto: Race Condition de Idempotência (Não Corrigido)

---

### Ø=ßà Severidade: ALTA — não corrigido nesta sessão, decisão do usuário pendente

Causa raiz: `MessageService#createFromWebhook` faz um `check-then-act` não atômico (`findFirst` seguido de `create`) e `Message.externalId` no `schema.prisma` tem apenas `@@index`, não `@@unique`. Sob concorrência real (2 execuções de `handleEvent` para o mesmo `externalId` rodando ao mesmo tempo), ambas passam pelo `findFirst` antes de qualquer uma `commitar` — nenhuma trava de banco impede a corrida.

Quando ocorre de verdade: o job do Bull estourar `WEBHOOK_TIMEOUT` enquanto `handleEvent` ainda está rodando (Bull não cancela a `promise` anterior, só para de esperar e agenda `retry`) faz duas execuções concorrentes da mesma mensagem. Uma 2ª réplica da API no futuro amplia a janela.

Correção recomendada (requer `migration`, por isso não aplicada nesta validação): (1) `@@unique([externalId])` no `model Message` — Postgres permite múltiplos `NULL`, mensagens de agente sem `externalId` não são afetadas; (2) trocar `findFirst+create` por `upsert()` ou tratar o erro P2002 (violação de `unique`) como "já existe".

## 10. Auditoria de Código (catch sem throw / Promise.catch / void async / return silencioso)

---

46 blocos catch revisados em todo apps/api/src (varredura repetida nesta sessão sobre o levantamento já feito na FASE 11 da sessão anterior). Nenhum outro caso crítico como o do B-39 original ou o achado da seção 8 foi encontrado.

| Local                                           | Padrão                                      | Severidade | Status                                      |
|-------------------------------------------------|---------------------------------------------|------------|---------------------------------------------|
| webhook.service.ts:105                          | catch sem throw                             | Crítica    | Corrigido (sessão B-39 original)            |
| webhook.processor.ts#onFailed                   | assunção não verificada                     | Crítica    | Corrigido nesta sessão                      |
| evolution.service.ts (3 métodos)                | catch{return null/false} sem log            | Baixa      | Não corrigido — fora do fluxo de<br>webhook |
| contact.service.ts#purgeMedia                   | catch por item, best-effort                 | Info       | Correto por design, documentado             |
| webhook.service.ts:180/244                      | .catch() fire-and-forget (mídia/<br>avatar) | Info       | Correto por design, documentado             |
| auto-attendance-engine/<br>conversation.service | job.remove().catch()==>undefined)           | Info       | Correto por design, documentado             |
| token-blacklist.service.ts                      | fail-open/fail-closed                       | Info       | Decisão de produto documentada              |

Nenhum "void async" encontrado.  
Nenhum "Promise.catch()" que  
descarta erro crítico sem justificativa  
foi encontrado fora dos itens acima.

## 11. Testes Automatizados (Resultado Final)

---

### Suíte

Backend — unitários/integração (npx jest)

Backend — E2E (npm run test:e2e)

Frontend (npm test)

Build (npm run build)

Lint (npm run lint:check)



## 12. Métricas e Tempos Coletados

### Medição

Recuperação de job travado (V1)  
Escrita em lote (addBulk, 1000 jobs)  
Processamento (1000 jobs, concorrência 1, sem I/O)  
Memória do worker (1.600 jobs processados)  
Memória do Redis (1.600 jobs)  
Outage de Redis simulada  
DLQ falsos positivos (bug da seção 8)

### Valor observado

~98s (kill -> stalled detectado ~26s -> reprocessa 25s do zero)  
~300-350ms (~2.900-3.250 jobs/s de escrita)  
~2-4s (~250-500 jobs/s)  
436MB -> 449MB (WS, cresce pouco, sem sinal de vazamento)  
4.32MB -> 4.79MB usados (pico 6.20MB)  
~42s (parada deliberada) — reconexão automática, sem restart manual  
9 (antes da correção) -> 0 (depois)

## 13. Riscos Residuais

- Race condition de idempotência sob concorrência real (Alta) — seção 9, requer migration.
- Producer sem fail-fast durante indisponibilidade do Redis — requisição pode ficar pendurada (Média).
- tenantId/conversationId/messageId ausentes nos logs de sucesso/retry, só no de falha definitiva (Média).
- 3 métodos de evolution.service.ts sem log em fallback silencioso (Baixa).
- Sem circuit breaker — resiliência é só por job, não agregada (Baixa/melhoria).
- Métricas em memória por réplica — não agregam entre múltiplas instâncias da API (já registrado como B-46).
- 5.000/10.000 jobs de carga não executados fisicamente, só extrapolados (limitação desta validação).

## 14. Classificação de Severidade Consolidada

| Severidade     | Item                                               |
|----------------|----------------------------------------------------|
| Crítica        | DLQ prematura em toda falha transitória            |
| Alta           | Race condition de idempotência sob concorrência    |
| Média          | Sem fail-fast do producer durante outage de Redis  |
| Média          | Campos de log incompletos fora do caminho de falha |
| Baixa          | 3 métodos silenciosos sem log em evolution.service |
| Baixa/Melhoria | Ausência de circuit breaker                        |

## 15. Recomendações e Melhorias Futuras

---

- Prioridade 1: @@unique([externalId]) em Message + upsert() em createFromWebhook (fecha a race de idempotência).
- Prioridade 2: timeout explícito no client Redis do Bull (fail-fast no producer durante outage).
- Prioridade 3: circuit breaker (cockatiel) em torno do EvolutionService.
- Prioridade 4: messageId em todos os logs (custo zero, só parsing); avaliar tenantId/conversationId também no sucesso (custo de 1-2 queries extras por job).
- Prioridade 5: logger.warn nos 3 catches silenciosos de evolution.service.ts.
- Futuro: prom-client real com histograma de latência (percentis); alerta automático sobre webhook\_dlq\_size; endpoint de reprocessamento em lote da DLQ.

## 16. Checklist Production Ready

---

### & Algum Job pode ser perdido?

! NÃO — comprovado em V1, V9, V10 (job sempre íntegro/persistido).

### & Algum Retry deixou de funcionar?

! NÃO — comprovado em V6/V7 após a correção (9/9 cenários recuperaram).

### & A DLQ está íntegra?

! SIM, após a correção do achado crítico (seção 8) — antes, não.

### & Existe risco de duplicidade?

! SIM — sob concorrência real (achado alto, seção 9), não sob entrega sequencial.

### & O Worker suporta restart?

! SIM — comprovado em V1 e V10 (job travado e restart completo).

### & O Redis suporta reconexão?

! SIM — comprovado em V9 (reconexão automática, sem intervenção).

### & O sistema suporta carga?

! SIM até ~2.600 jobs testados; acima disso, extrapolado, não medido.

### & Existem gargalos?

! SIM — concorrência 1 por processo é o teto real (achado arquitetural, não bug).

### & Existem riscos residuais?

! SIM — ver seção 13 (1 alto, 2 médios, 2 baixos).

### & A arquitetura pode ser considerada Production Ready?

! COM RESSALVAS — ver conclusão final.

## 17. Conclusão Final e Aprovação da Arquitetura

---

O objetivo original do B-39 — nenhuma mensagem perdida silenciosamente, retry automático funcional, backoff exponencial, DLQ íntegra — está comprovadamente atingido para os cenários testados nesta sessão, incluindo cenários que exigem infraestrutura real e não podem ser verificados só por teste unitário (graceful shutdown, indisponibilidade e restart do Redis, saturação de fila).

Esta validação teve valor além de confirmar o que já existia: encontrou e corrigiu, ao vivo, um bug crítico (DLQ prematura) que nenhuma suíte de testes anterior detectava, porque nenhum teste unitário exercitava a interação real entre "retry que ainda vai acontecer" e o evento "failed" do Bull disparando a cada tentativa. Isso demonstra o valor concreto de validar contra infraestrutura real em vez de apenas mocks.

Por outro lado, a mesma validação encontrou uma lacuna de idempotência sob concorrência real que não estava coberta nem pelos testes unitários nem pela validação sequencial anterior — e que só se torna um risco genuíno PORQUE o retry agora funciona de verdade (antes do B-39, sem retry, essa concorrência nunca acontecia). É uma dívida técnica nova, descoberta como consequência direta e esperada de corrigir o bug original — não uma regressão introduzida por esta sessão.

### Ø=βá VEREDITO: APROVADO COM RESSALVAS

A arquitetura de webhooks pode ir para produção com o entendimento explícito de que existe uma janela de concorrência estreita (mas real) que pode duplicar uma mensagem e seus efeitos colaterais. Recomenda-se fechar o achado de severidade Alta (seção 9) — migration simples e de baixo risco — antes do primeiro cliente com volume relevante, mas isso não bloqueia o uso atual em produção de baixo volume, onde a janela de corrida é rara o suficiente para não ter se manifestado até hoje.

Relatório gerado a partir de execução real contra Docker/Redis/Postgres locais — não simulado. Evidências brutas (logs, timestamps, queries) preservadas em apps/api/tmp-validation/evidence.md durante a sessão.